RhymeFlow: Training-Free Acceleration for Video Generation with Asynchronous Denoising Flow Scheduling

Chensheng Dai[1,*], **Shengjun Zhang**[1,*], **Yifan Li**[1], **Zhang Zhang**[1], **Zheng Zhu**[2], **Yueqi Duan**[1,†]

[1]Tsinghua University, [2]GigaAI

Abstract

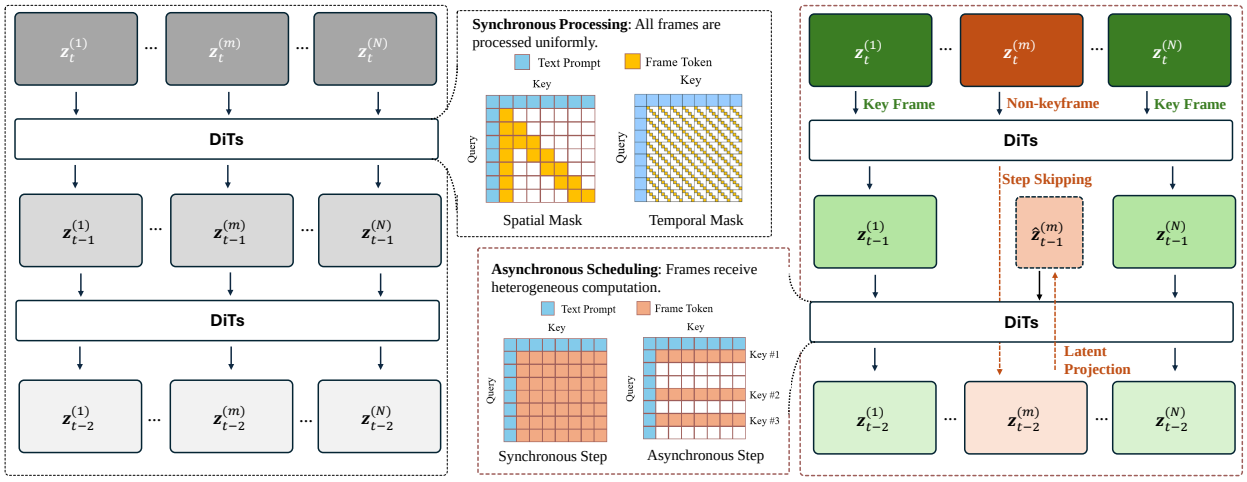
Video generation models based on Diffusion Transformers (DiTs) have achieved remarkable performance in video synthesis, yet they suffer from high inference latency and computational costs due to the quadratic complexity of 3D attention. Existing acceleration methods primarily reduce computational complexity within each individual denoising steps through techniques such as sparse attention and KV-caching. However, they rigidly adhere to the inherent constraint of the standard diffusion pipeline: every frame in the target video sequence must be subjected to a complete, dense denoising process across all diffusion timesteps. We observe that due to the corresponding contents and motions among adjacent frames, when keyframes with critical semantic transitions are anchored, the intermediate states of others often follow more predictable trajectories, which indicates that such uniform, dense denoising process is inherently redundant for natural video data. To this end, we introduce **RhymeFlow**, a training-free framework that decouples the denoising trajectories of different frames. Specifically, we first identify a sparse set of pivotal key frames that dominate the latent semantic evolution. Then, only these keyframes undergo dense, step-by-step denoising to ensure structural integrity, while non-keyframes progressively skip denoising steps to minimize computational cost. Since skipped intermediate states of non-keyframes break the temporal coherence in keyframe denoising steps, leading to visual degradation, we further introduce a latent trajectory projection module, which enables keyframes to interact with a complete and temporally consistent sequence representation. Extensive experiments on current DiT-based video generation models demonstrate our method outperforms existing baselines with higher inference speed and better visual quality.

GitHub Repo: https://github.com/Simon-Dcs/RhymeFlow

1 Introduction

Video diffusion models [5, 12, 39], particularly those based on DiT architectures [10, 26, 28], have achieved remarkable success in high-fidelity video synthesis. However, their practical deployment is hampered by a inherent bottleneck: the quadratic complexity of 3D spatiotemporal attention combined with dozens of denoising steps creates prohibitive inference costs [7, 19]. To mitigate this, training-free acceleration methods [24, 33, 42] have garnered significant attention due to their compatibility with pre-trained models, which broadly fall into three categories: KV-cache management [16, 25, 27], model compression via quantization or token pruning [2, 8, 44], and sparse attention mechanisms [36, 37, 40]. While these approaches effectively lower the complexity of every individual denoising step, every video frame is subjected to a dense, stepwise

∗ Equal contribution, † Corresponding author



(a) Conventional Synchronous Scheduling

(b) **RhymeFlow**: Asynchronous Denoising Scheduling

Figure 1 Different sparse attention-based acceleration methods. (a) Synchronous Scheduling: Conventional acceleration methods operate within a synchronous framework where all frames are jointly denoised at each timestep. These approaches achieve efficiency by exploiting intra-step sparsity through techniques such as spatial, temporal, or semantic attention masking. (b) Our Asynchronous Scheduling: Our work introduces an orthogonal acceleration dimension. Instead of processing all frames uniformly, we differentiate between keyframes and non-keyframes. Keyframes undergo a full, step-by-step denoising process to preserve high fidelity, while non-keyframes are updated asynchronously, effectively skipping computation at certain timesteps.

denoising procedure across the full set of diffusion timesteps.

In this work, we observe that such uniform computational allocation is inherently redundant. Due to the overlapping visual contents and continuous motion trajectory among neighbor frames, video sequences natively exhibit strong spatiotemporal coherence. Building upon this, we find that once a sparse set of pivotal keyframes—those capturing critical structural or semantic transitions—is anchored via step-by-step full attention, the intermediate states of adjacent non-keyframes often follow highly predictable trajectories in the latent space. Driven by this predictability, allowing non-keyframes to appropriately skip certain denoising steps does not incur noticeable degradation in visual quality. This observation indicates that applying a uniform, dense denoising schedule to every single frame is fundamentally unnecessary. Consequently, designing heterogeneous, frame-specific denoising schedules presents a promising, yet largely untapped, potential for video diffusion acceleration.

To this end, we introduce **RhymeFlow**, a training-free framework that establishes a new paradigm for video diffusion acceleration with **Asynchronous Denoising Flow Scheduling**. Our core insight is to decouple the denoising trajectories of individual frames and assign heterogeneous schedules accordingly. Specifically, we first identify a sparse set of pivotal keyframes by detecting semantic shifts in the latent space, ensuring that frames capturing critical transitions are prioritized for high-fidelity generation. Second, we assign heterogeneous denoising schedules: only these keyframes execute the complete, step-by-step denoising process to ensure structural integrity and high visual fidelity. In contrast, non-keyframes skip designated denoising steps to minimize computational costs. Third, recognizing that skipped intermediate states invariably break the temporal coherence required for 3D attention, we introduce a lightweight latent trajectory projection module. This mechanism analytically estimates the skipped intermediate latents of non-keyframes, which ensures keyframes can attend to a complete and temporally consistent sequence representation at every step, thereby preserving global consistency without incurring the penalty of full network evaluation.

2 Related Works

2.1 Video Diffusion Models

Recent years have witnessed the remarkable success of diffusion models [11, 35] in generating high-fidelity and diverse content. This paradigm has been successfully extended from images to the video domain, with Diffusion Transformers (DiTs) [10, 26, 28] emerging as the dominant architecture. State-of-the-art open-source models like Wan 2.1 [34] , CogVideoX [13, 41], and HunyuanVideo [18], as well as closed-source systems like Sora [3] and Kling, have demonstrated unprecedented capabilities in generating temporally consistent and visually stunning videos from text or image prompts. These models typically adapt the 2D spatial attention mechanism to a more computationally intensive 3D spatiotemporal attention to capture both the appearance within frames and the motion across them [1, 18, 41].

However, this success comes at a great cost. The inference process of these models is notoriously slow, often requiring hundreds of sequential passes through a massive transformer network. The 3D attention mechanism, in particular, exhibits quadratic complexity with respect to the number of tokens (pixels $\times$ frames), making the generation of long, high-resolution videos prohibitively expensive. This significant computational barrier motivates a strong need for efficient video generation techniques.

2.2 Efficient Video Generation

To combat the prohibitive inference costs, researchers have explored several avenues for acceleration. We categorize these efforts and position our work in relation to them.

Decreasing the Denoising Steps. A primary strategy for accelerating diffusion models is to reduce the number of function evaluations (NFE) required during sampling. Early models based on stochastic differential equations (SDEs) required thousands of steps. The introduction of ordinary differential equation (ODE) solvers, such as Denoising Diffusion Implicit Models (DDIM) [31], and more advanced solvers like DPM-Solver [22, 23, 48], significantly reduced the required steps. More recently, methods like Consistency Models [32] and progressive distillation [30] have been proposed to enable high-quality synthesis in just a few steps. However, these approaches have two major limitations in the context of large-scale video models. First, many of them, such as distillation and consistency training, require costly retraining or fine-tuning, which is impractical for models with billions of parameters. Second, they apply a uniform step-reduction strategy to all frames, treating them as equally important throughout the denoising process.

Diffusion Model Compression. An orthogonal line of research focuses on compressing the model itself to reduce the computational cost of each forward pass. This includes quantization techniques that reduce the bit-width of weights and activations, such as the W8A8 strategy in Q-Diffusion [21], or even more aggressive 4-bit schemes [20]. Other approaches involve designing more efficient model architectures from the ground up or using more compact autoencoders.

Training-free Acceleration via Caching. A recent and related trend in training-free acceleration involves caching and reusing intermediate results. Methods like DeepCache [27] and FasterCache [25] leverage the high feature similarity between adjacent denoising steps, avoiding redundant computation by reusing cached features (e.g., from the UNet's deeper layers or KV-cache). These methods are training-free and effectively reduce computation.

Sparse Attention in LLMs and Video. The quadratic complexity of the attention mechanism has motivated extensive research into sparse attention, particularly in Large Language Models (LLMs). Methods like StreamingLLM [38] and H2O [46] identify that attention scores are often concentrated on a small subset of "heavy-hitter" or local tokens. This concept has been adapted for video diffusion. For instance, the proposed frameworks SVG [36] and SAP [40], perform an in-depth analysis of attention patterns in Video DiTs, classifying heads into Spatial and Temporal types and applying structured sparse masks accordingly. While highly effective, these methods focus on optimizing the computation within a single denoising step by reducing the number of token-to-token interactions. Instead of making the attention matrix sparse, RhymeFlow skips the entire forward pass for certain frames at certain steps, optimizing the denoising trajectory over time. This makes our asynchronous denoising flow scheduling paradigm orthogonal to intra-step sparsity.

3 Method

In this section, we preform a detailed introduction of **RhymeFlow**, a training-free acceleration framework for video diffusion models. Our approach re-envisions the denoising process by assigning different denoising schedule to different frames. We first classify keyframes and non-keyframes based on the latent semantic evolution. After the warm-up denoising stages, we propose a progressive skip strategy, where non-keyframes skip less steps at high noise levels and skip more steps at low noise levels. To maintain the temporal coherence of natural video data, we further present a latent trajectory projection mechanism to enable keyframes to interact with non-keyframes at the denoising steps that they skip.

3.1 Sequential Keyframe Selection

To alleviate the prohibitive inference latency and excessive computational cost of prevailing video generation models while preserving the temporal continuity and semantic integrity of the generated sequence, we design a lightweight, content-aware sequential key frame selection scheme to identify representative frames that dominate the semantic transitions of the entire video.

Supposing that the video representation in the diffusion latent space consist of N latent frames, we denote the noisy latent of the i-th frame at the current denoising timestep t as $\mathbf{z}_t^{(i)}(1 \leq t \leq N)$. Since the raw noisy latents are heavily corrupted by noise and lack distinct structural information, we perform a single-step denoising prediction to estimate the fully denoised clean latent, denoted as $\hat{\mathbf{z}}_0^{(i)}$. These predicted clean latents provide a structurally clearer and more robust proxy for evaluating frame-wise correlations.

Based on the clean latent frames, we conduct the selection procedure in a strictly sequential manner along the temporal axis of the target video. Previous studies [6, 9, 17, 29] have comprehensively demonstrated that the initial frame of a video sequence acts as the fundamental semantic and visual anchor for subsequent frame synthesis, and plays an irreplaceable role in maintaining long-range temporal consistency and semantic fidelity throughout the generated video. In light of this, we initialize the set of selected key frames $\mathcal{K} = \{\hat{\mathbf{z}}_0^{(i)}\}$ with the initial frame of the target sequence. Subsequently, we traverse all subsequent candidate frames in chronological order. For each candidate latent frame $\hat{\mathbf{z}}_0^{(t)}$, we compute the pairwise similarity $\text{sim}(\hat{\mathbf{z}}_0^{(t)}, \hat{\mathbf{z}}_0^{(k)})$ between the candidate frame and the identified nearest preceding key frame $\hat{\mathbf{z}}_0^{(k)} \in \mathcal{K}$. The update rule of the key frame set is written as:

$$\mathcal{K} \leftarrow \begin{cases} \mathcal{K} \cup \{\hat{\mathbf{z}}_0^{(t)}\}, & \text{if } \psi_{\text{sim}}(\hat{\mathbf{z}}_0^{(t)}, \hat{\mathbf{z}}_0^{(k)})) < \tau \\ \mathcal{K}, & \text{otherwise} \end{cases} \tag{1}$$

where ψ_{sim} is defined as the cosine similarity function, and τ is a pre-defined threshold.

3.2 Asynchronous Denoising Flow Scheduling

We design a heterogeneous denoising schedule where keyframes and non-keyframes are updated at different time-steps.

3.2.1 Initial Synchronous Warm-up

The generation process commences with a synchronous warm-up stage that spans the initial T_w denoising steps. This foundational phase is critical, as the early steps of the diffusion process are primarily responsible for establishing the global compositional structure, motion priors, and color palette of the nascent video. Bypassing full computation during this period can introduce severe, often irrecoverable, artifacts and quality degradation. Consequently, for the initial T_w steps (i.e., from timestep $t = T$ down to $t = T - T_w$), all N latent frames are updated simultaneously. The update for each latent frame $\mathbf{z}_t^i$ to $\mathbf{z}_{t-1}^i$ proceeds using the standard, unmodified denoising function with full attention, as illustrated in Figure 2 (a).

3.2.2 Progressive Asynchronous Scheduling

The reverse diffusion process exhibit strong non-uniformity along the timestep axis. The early stages, characterized by high noise levels (large t), are dedicated to establishing the fundamental, low-frequency

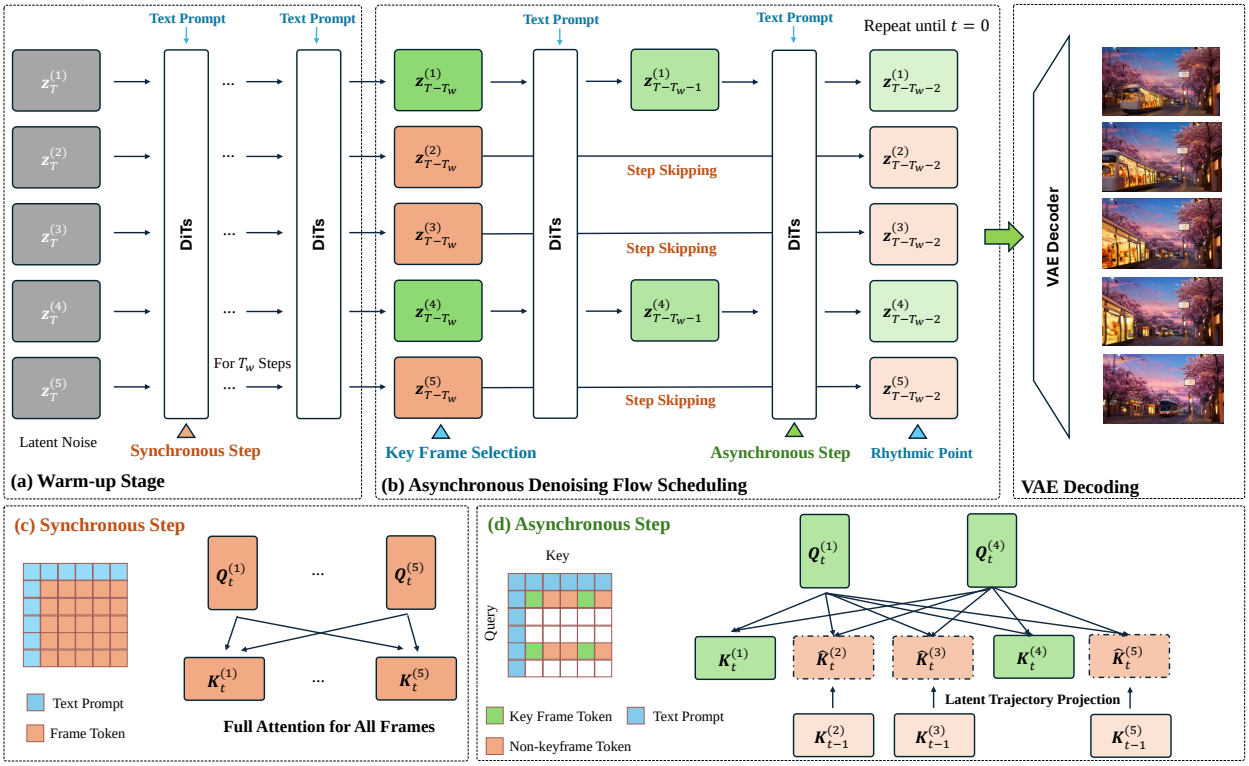


Figure 2 Pipeline. (a) **Warm-up Stage:** All frames are processed uniformly via synchronous updates for the initial T_w steps. (b) **Asynchronous Scheduling:** After warm-up, keyframes are selected. The pipeline transitions to an asynchronous schedule: keyframes are fully updated to preserve critical details, while non-keyframes are sparsely processed to accelerate generation. (c) **Synchronous Step:** A standard full-attention mechanism is applied during warm-up and at "Rhythmic Points" to synchronize all latent frame representations. (d) **Asynchronous Step:** We introduce latent trajectory projection to efficiently estimate the skipped states of non-keyframes, providing the temporal context required for keyframes to attend to the full sequence.

structure of the video, such as global composition and object forms. In this phase, the denoising trajectory is highly sensitive and less predictable, making these steps foundational to the final output quality. Conversely, the later stages (small t) primarily involve the refinement of high-frequency details, where the latent trajectory becomes significantly smoother and thus more predictable.

A static step-skipping stride cannot adapt to optimally navigate this evolving dynamic. An aggressive stride risks irrecoverable damage to the foundational structure in the early stages, while a conservative stride would forgo significant efficiency gains in the predictable later stages. To strike a deliberate balance between computational efficiency and generative fidelity, we introduce a Progressive Asynchronous Scheduling strategy. This approach adaptively increases the skipping stride for non-keyframes as the denoising process advances.

We implement this strategy with a simple, yet effective, piecewise schedule for the skip stride, n_{skip}, as a function of the current timestep t:

$$n_{\text{skip}}(t) = \begin{cases} n_{\text{small}}, & \text{if } T_{\text{mid}} < t \leq T - T_w \\ n_{\text{large}}, & \text{if } t \leq T_{\text{mid}} \end{cases} \tag{2}$$

where T_{mid} is a mid-point hyperparameter (e.g., $T/2$), and $n_{\text{small}} < n_{\text{large}}$ are integer stride values (e.g., $n_{\text{small}} = 2$, $n_{\text{large}} = 3$). This schedule intelligently allocates computational resources where they are most critical, thereby achieving a near-optimal trade-off between acceleration and the perceptual quality of the final generated video.

3.2.3 Asynchronous Denoising Step

Following the warm-up stage, RhymeFlow transitions into an asynchronous denoising schedule characterized by heterogeneous computational trajectories for different frame categories:

- **Keyframes** are updated via a standard single-step denoising process, advancing their state from $\mathbf{z}_t^{(k)}$ to $\mathbf{z}_{t-1}^{(k)}$. This ensures they maintain fidelity.
- **Non-keyframes** are projected forward multiple steps in a single computation. Their state is advanced directly from $\mathbf{z}_t^{(k)}$ to $\mathbf{z}_{t-n_{\text{skip}}}^{(k)}$, thereby bypassing the intermediate denoising steps and accelerating the generation process.

Since keyframes and non-keyframes progress with staggered strides, they periodically coincide at specific timesteps, which we define as *Rhythmic Points*. At these points, the framework executes a **full synchronous update**, wherein the latent representations of all frames engage in global 3D attention. This periodic re-synchronization provides a comprehensive contextual foundation, critical for maintaining global video coherence. Moreover, Rhythmic Points function as computational checkpoints where the trajectories of non-keyframes are recalibrated against the high-fidelity keyframe anchors. Such a mechanism effectively bounds the error accumulation inherent in prolonged step-skipping, ensuring that acceleration does not compromise temporal consistency or perceptual quality.

However, a fundamental challenge arises during the intermediate steps $\tau \in [t - n_{\text{skip}} + 1, t - 1]$, a keyframe $\mathbf{z}_\tau^{(k)} \in \mathcal{K}$ must be updated. This update requires attention context from all other frames at the target timestep τ. The central challenge arises here: a non-keyframe $\mathbf{z}_\tau^{(j)}$ has been fast-forwarded to $t - n_{\text{skip}}$, meaning its latent states for any intermediate timestep τ are not explicitly computed and do not exist.

3.2.4 Latent Trajectory Projection

Inspired by the linear nature of ODE trajectories in Rectified Flow, we analytically project an approximation of the missing latent state. For a non-keyframe j that was last updated at t_{start} to obtain $\mathbf{z}_{t_{\text{end}}}^{(j)}$, we can project its latent state $\hat{\mathbf{z}}_\tau^{(j)}$ at the intermediate time t via linear interpolation in the latent space:

$$\hat{\mathbf{z}}_\tau^{(j)} = (1 - \alpha) \cdot \mathbf{z}_{t_{\text{start}}}^{(j)} + \alpha \cdot \mathbf{z}_{t_{\text{end}}}^{(j)}, \quad \alpha = \frac{t_{\text{start}} - \tau}{t_{\text{start}} - t_{\text{end}}} \tag{3}$$

This projection is computationally trivial yet provides a robust estimate of the intermediate state. Following this latent projection, we can then compute the required Key and Value vectors, $\hat{\mathbf{K}}_\tau^{(j)}$ and $\hat{\mathbf{V}}_\tau^{(j)}$, on-the-fly.

With this mechanism, we define the logic for a keyframe update: it attends to the true states of other keyframes and the projected states of non-keyframes. A non-keyframe only performs its update at its designated, less frequent steps.

3.2.5 KV-Cache Management.

Our asynchronous denoising schedule necessitates a specialized KV-cache mechanism fundamentally distinct from causal autoregressive approaches. We introduce a **per-layer rolling cache** $\mathcal{C}_\ell^{(f)}$ for each DiT layer ℓ that stores the two most recent attention outputs for each non-keyframe $\hat{\mathbf{z}}$: $\mathbf{h}_{t_1}^{(\ell,\hat{\mathbf{z}})}$ and $\mathbf{h}_{t_2}^{(\ell,\hat{\mathbf{z}})}$ at timesteps $t_1 > t_2$, along with their temporal indices. Critically, we cache the post-attention hidden states rather than input key-value pairs, as these representations have already integrated full-frame contextual information. When a keyframe requires context from a non-keyframe at a skipped timestep $\tau \in (t_1, t_2)$, we employ latent trajectory projection. These interpolated representations serve as transient key-value providers for the current attention operation and are immediately discarded thereafter, eliminating persistent storage of intermediate states.

4 Experiments

Table 1 Quantitative evaluation of the efficiency-fidelity trade-off on Wan 2.1. We investigate the impact of warm-up timesteps (T_w) and the number of keyframes (M). Our optimal default configuration is highlighted in bold.

Method		Similarity Metrics			Video Quality		Efficiency	
T_w	M	PSNR ↑	SSIM ↑	LPIPS ↓	SubCon. ↑	ImgQual. ↑	Lat.(s) ↓	Speedup ↑
Original		-	-	-	0.9102	0.6946	993.5	-
6	3	20.998	0.615	0.206	0.8460	0.6099	607.2	1.64×
6	4	22.588	0.617	0.196	0.8478	0.6362	642.0	1.55×
6	5	25.669	0.657	0.184	0.8536	0.6883	682.0	1.46×
8	3	23.742	0.721	0.182	0.8601	0.6344	621.6	1.60×
8	**4**	**26.291**	**0.783**	**0.168**	**0.8831**	**0.6706**	**650.4**	**1.53**×
8	5	27.707	0.812	0.162	0.8896	0.6902	712.8	1.39×
10	3	24.006	0.742	0.169	0.8818	0.6714	630.8	1.57×
10	4	26.636	0.817	0.164	0.8838	0.6819	670.3	1.48×
10	5	28.061	0.816	0.161	0.8911	0.6919	738.1	1.35×

4.1 Experimental Settings

Models. To validate the effectiveness of our proposed framework, we integrate it into two prominent open-source text-to-video (T2V) generation models: Wan2.1-T2V-v1.3B-Diffusers [34] and CogVideoX-v1.5-T2V [41]. Both models are configured to produce videos of 81 frames at 720p resolution. The architectural specifics of these models are noteworthy: in its latent space, Wan 2.1 operates on 21 latent frames, with each frame represented by 3600 tokens post 3D-VAE processing. In contrast, CogVideoX-v1.5 utilizes a 3D full attention mechanism over 11 frames, where each frame corresponds to 4080 tokens after its 3D-VAE.

Metrics. We evaluate our method across two primary dimensions: generation fidelity and computational efficiency. To quantify fidelity preservation, we compare the accelerated outputs against the unaccelerated baselines using standard pixel-level and perceptual metrics: Peak Signal-to-Noise Ratio (PSNR), Structural Similarity (SSIM), and Learned Perceptual Image Patch Similarity (LPIPS). Additionally, we assess the visual quality of the generated videos using VBench [14], focusing on critical aspects such as subject consistency and imaging quality. Finally, to demonstrate computational efficiency, we report the average inference latency of diffusion process alongside the corresponding speed-up ratio.

Baselines. To contextualize the performance of RhymeFlow, we conduct a comparative analysis against several state-of-the-art (SOTA) training-free acceleration methods. Our baselines include SpargeAttn [45], MInference [15], Pyramid Attention Broadcast (PAB) [47], Sparse VideoGen (SVG) [36], and Semantic-Aware Permutation (SAP) [40]. For a fair comparison, we adopt the official hyperparameter configurations provided by the authors for their respective text-to-video generation tasks. All experiments are executed on a single NVIDIA A800 GPU to ensure a consistent hardware environment.

Implementation Details. Our text-to-video generation experiments utilize prompts sourced from the Penguin Benchmark, which have been refined through the prompt optimization process detailed in VBench [14]. The core hyperparameters are configured as follows: the duration of the warm-up stage, T_w, is set to 8 steps, and the total number of keyframes, M, is 4. For the progressive asynchronous scheduling, the update strides are defined as $n_{\text{small}} = 2$ and $n_{\text{large}} = 3$.

4.2 Trade-off Analysis of Scheduling Parameters

To evaluate our asynchronous scheduling, we analyze the efficiency-fidelity trade-off by varying the warm-up duration (T_w) and keyframe budget (M). As shown in Table 1, both parameters predictably trade acceleration for generation quality.

Specifically, extending T_w strengthens the global structure, steadily improving Subject Consistency (SubCon.)

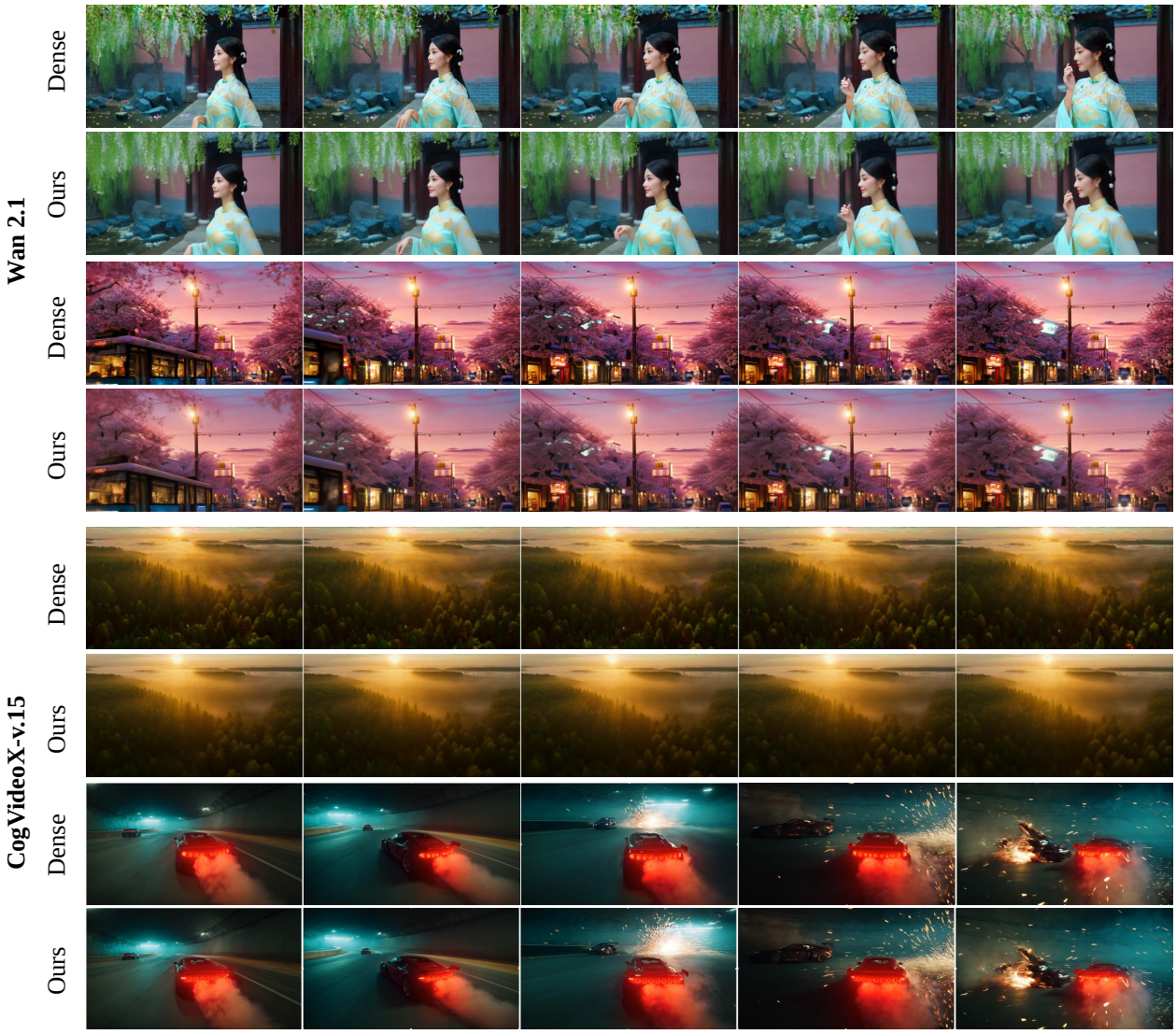


Figure 3 Qualitative visual comparisons. Frame sequences generated by the standard dense attention baseline versus our RhymeFlow. The top two examples are produced by Wan 2.1, while the bottom two are from CogVideoX-v1.5.

and Image Quality (ImgQual.). Concurrently, increasing M provides more frequent high-fidelity temporal anchors, which effectively reduces error accumulation; for instance, at a fixed $T_w = 8$, raising M from 3 to 5 boosts PSNR from 23.742 to 27.707 and SSIM from 0.721 to 0.812, though the inference speedup drops from 1.60× to 1.39×.

Empirically, we identify $T_w = 8$ and $M = 4$ as the optimal configuration. This setting preserves strong visual and temporal fidelity (ImgQual. 0.6706, SubCon. 0.8831) comparable to the original dense model, while delivering a substantial 1.53× inference speedup (650.4s vs. 993.5s). Consequently, we adopt this configuration as our default for all subsequent benchmark comparisons.

4.3 Comparison with State-of-the-Art Methods

We benchmark RhymeFlow against several state-of-the-art training-free video acceleration methods across two fundamentally different architectures: Wan 2.1 and CogVideoX-v1.5. To ensure a fair and rigorous evaluation, we select baseline methods based on their official architectural compatibility. For instance, SAP

Table 2 Quality and efficiency benchmarking on Wan 2.1 and CogVideoX-v1.5.

Method	Quality					Efficiency	
	PSNR ↑	SSIM ↑	LPIPS ↓	SubCon. ↑	ImgQual. ↑	Lat.(s) ↓	Speedup ↑
Wan 2.1	-	-	-	0.9102	0.6946	993.5	-
SpargeAttn [45]	20.399	0.613	0.393	0.8632	0.7118	719.7	1.38×
SVG [36]	22.419	0.694	0.290	0.8758	0.6913	708.0	1.40×
SAP [40]	24.454	0.730	0.223	0.8789	0.6837	608.5	1.63×
Ours	**26.291**	**0.783**	**0.168**	**0.8831**	**0.6706**	**650.4**	**1.53×**
Ours + SAP	**24.586**	**0.737**	**0.221**	**0.8792**	**0.6806**	**596.8**	**1.66×**
CogVideoX-v1.5	-	-	-	0.987	0.624	625.0	-
MInference [15]	22.490	0.743	0.264	0.874	0.589	422.3	1.48×
PAB [47]	23.230	0.782	0.145	0.978	0.573	443.3	1.41×
SVG [36]	24.130	0.811	0.171	0.982	0.597	385.8	1.62×
Ours	**26.890**	**0.852**	**0.142**	**0.986**	**0.623**	**351.1**	**1.78×**
Ours + SAP	**25.574**	**0.821**	**0.157**	**0.972**	**0.609**	**323.8**	**1.93×**

Table 3 Quality and efficeincy benchmarking on HunyuanVideo.

Method	PSNR ↑	SSIM ↑	LPIPS ↓	Latency (s) ↓	Speedup ↑
SVG [36]	21.17	0.684	0.392	3459	1.92 ×
SAP [40]	24.64	0.904	0.068	2634	2.52 ×
EasyCache [49]	23.51	0.861	0.119	2850	2.33 ×
DiCache [4]	23.54	0.860	0.114	2814	2.36 ×
VGDFR [43]	19.49	0.779	0.199	3019	2.20 ×
Ours	**26.34**	**0.918**	**0.060**	2939	2.26 ×
Ours+SAP	25.01	0.910	0.068	**2555**	**2.60** ×

[40] is evaluated strictly on Wan 2.1, while MInference [15] and PAB [47] are evaluated on CogVideoX-v1.5, mitigating any performance degradation caused by unverified cross-architecture adaptation.

4.3.1 Quantitative Evaluation

As detailed in Table 2, RhymeFlow consistently establishes a superior trade-off between generation quality and efficiency. On Wan 2.1 [34], RhymeFlow achieves a high PSNR of 26.291 and an SSIM of 0.783 with a 1.53× speedup, significantly outperforming intra-step methods like SpargeAttn [45] and standalone SVG [36] in visual fidelity. This advantage is equally evident on CogVideoX-v1.5 [41], where RhymeFlow attains a leading 1.78× speedup while preserving a remarkable SubConsistency of 0.986. We further benchmark against EasyCache [49], DiCache [49], and VGDFR [43] on high-dynamic HunyuanVideo [18] clips in Table 9. Our base method achieves the best visual quality across all metrics, outperforming cache-based methods, such as EasyCache [49] and DiCache [4]. When combined with SAP (Ours+SAP), our approach achieves the lowest latency and highest speedup.

4.3.2 Orthogonality and Synergistic Combination

As discussed in Section D.1, our asynchronous frame-scheduling operates inter-step, making it inherently orthogonal to token-level (intra-step) sparse attention methods. To demonstrate this, we evaluate a hybrid approach, denoted as *Ours + SAP*. Notably, standard SAP imposes an aggressive sparsity ratio of 0.3. Because RhymeFlow already significantly reduces the computational burden by skipping frame-level updates, applying extreme token-level sparsity concurrently would lead to information bottleneck. Therefore, we

Table 4 Ablation on key frame selection strategy.

Setup	PSNR ↑	SSIM ↑	LPIPS ↓	Latency ↓	Speedup ↑
Original	-	-	-	993.5	-
Random	20.630	0.525	0.383	650.0	1.53×
First	19.220	0.515	0.402	651.0	1.53×
Uniform	24.293	0.643	0.183	649.0	1.53×
Ours	26.291	0.783	0.168	650.4	1.53×

Table 5 Ablation study on Architectural Components.

Method	PSNR ↑	SSIM ↑	LPIPS ↓	Latency ↓	Speedup ↑
Original	-	-	-	993.5	-
w/o Progressive	25.399	0.753	0.172	708.0	1.40×
w/o Projection	20.630	0.525	0.383	622.2	1.60×
Ours	26.291	0.783	0.168	650.4	1.53×

deliberately relax the SAP [40] sparsity ratio to 0.5 in the hybrid setting. This synergistic parameter scheduling allows *Ours + SAP* to not only achieve the fastest generation speeds (1.66× on Wan 2.1 [34] and 1.93× on CogVideoX-v1.5 [41]) but also yield better generation quality than the standalone SAP [40] baseline.

4.3.3 Qualitative Results

The quantitative superiority translates directly into visual fidelity. As illustrated in Figure 3, we present qualitative comparisons between the standard dense baseline and RhymeFlow across a diverse array of challenging scenarios. From top to bottom, these include fine-grained human portraits, structurally complex urban cityscapes, nuanced natural environments, and high-dynamic motion scenes. Across all settings, RhymeFlow perfectly maintains the intricate textures, lighting consistency, and complex motion dynamics of the dense baseline without introducing the flickering or blurring artifacts commonly associated with aggressive acceleration strategies.

4.4 Ablation Study

To thoroughly validate the design choices of RhymeFlow, we conduct extensive ablation studies, analyzing keyframe selection, core framework components, and memory management on Wan 2.1 [34].

4.4.1 Ablation on Key Frame Selection Strategy

We first evaluate our content-aware keyframe selection mechanism against three heuristic alternatives with a fixed budget of $M = 7$: *Random*, *First* (initial M frames), and *Uniform* (evenly spaced intervals).

As presented in Table 4, naive placement strategies like *Random* and *First* lead to severe visual degradation (PSNR drops below 21). The *Uniform* strategy serves as a much stronger baseline by providing a basic temporal coverage, achieving a PSNR of 24.293. However, because it blindly assigns keyframes without considering the actual semantic shifts within the video, it struggles to maintain high-frequency details across dynamic scenes. By dynamically identifying frames with substantial latent changes, our method significantly outperforms the *Uniform* baseline, boosting SSIM from 0.643 to 0.783 and lowering LPIPS from 0.183 to 0.168. This highlights that an optimal strategy must be context-aware, placing high-fidelity anchors exactly where motion or semantic transitions occur.

Table 6 Ablation study on KV-Cache management.

Method	Latency (s) ↓	Speedup ↑	Average Memory (GB) ↓	Peak Memory (GB) ↓
Original	993.5	1.00×	24.4	44.3
w/o KV-Cache	653.6	1.52×	27.1	49.9
Ours	650.4	1.53×	21.5	42.6

Table 7 Latent-state analysis against dense sampler on Wan 2.1.

Variant	PSNR↑	$\mathcal{E}_{\text{keyframe}}$ ↓	$\mathcal{E}_{\text{non-keyframe}}$ ↓	$\mathcal{E}_{\text{projected-nonkeyframe}}$ ↓	Speedup↑
Ours (full)	**27.70**	**0.0019**	**0.0022**	**0.0018**	1.44×
w/o latent projection	24.44	0.0190	0.0029	0.0240	1.55×
w/ keyframe-only attention	24.37	0.0043	0.0045	0.0490	**1.69**×

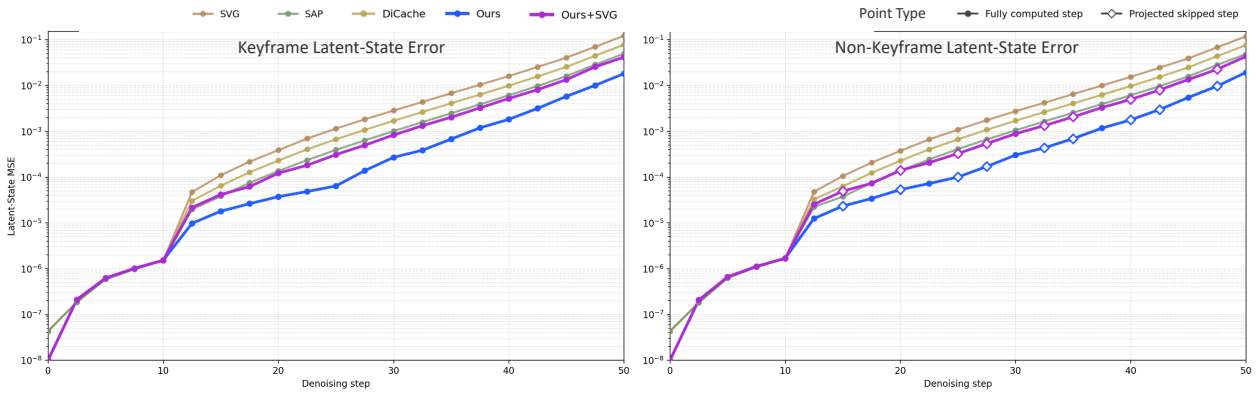


Figure 4 The error analysis of the latent projection on high-dynamic videos.

4.4.2 Ablation on Core Architectural Components

Table 5 isolates the contributions of our core algorithmic designs. Replacing progressive scheduling with a fixed-step skip (*w/o Progressive*) diminishes both acceleration (1.53× → 1.40×) and visual fidelity (PSNR 26.291 → 25.399). This validates our dynamic stride, which optimally preserves dense updates in noise-sensitive early stages while aggressively skipping predictable later stages. Conversely, omitting trajectory projection (*w/o Projection*) forces keyframes to attend only to the sparse synchronization group. Although this yields a marginal speedup (1.60×), depriving the model of dense temporal context triggers a quality collapse (PSNR plummets to 20.630). Thus, this computationally lightweight projection is indispensable for stabilizing asynchronous diffusion.

4.4.3 Efficacy of KV-Cache Management

Table 6 demonstrates that our KV-Cache management is essential for suppressing memory overhead. While both configurations (*w/o KV-Cache* and *Ours*) achieve similar speedups (1.52× vs. 1.53×) natively driven by asynchronous step-skipping, the naive implementation (*w/o KV-Cache*) must retain historical intermediate states to perform future latent projections. This persistent storage severely bloats the peak VRAM to 49.9 GB, exceeding even the original dense model (44.3 GB). By introducing a per-layer rolling cache that instantly discards transient interpolated features, RhymeFlow efficiently bounds the peak memory to 42.6 GB. This deliberate design ensures accessible GPU deployment without sacrificing acceleration.

Table 8 Double-blind user study results (%).

Metric	Ours v.s. SVG				Ours v.s. SAP				Ours v.s. Dense			
	Ours	Tie	Opp.	p-Value	Ours	Tie	Opp.	p-Value	Ours	Tie	Opp.	p-Value
Visual Quality	**51.2**	18.3	30.5	.025	**74.4**	14.6	11.0	<.001	18.3	**53.7**	28.0	.256
Temporal Coherence	**53.7**	25.6	20.7	<.001	**78.0**	12.2	9.8	<.001	18.3	**58.5**	23.2	.608
Perceptual Preference	**51.2**	23.2	25.6	.006	**79.3**	12.2	8.5	<.001	15.9	**56.1**	28.0	.133

4.4.4 Latent Projection Analysis

The near-straight per-frame trajectories induced by flow matching keep the linear-projection error small throughout denoising. Figure 4 plots per-step latent MSE, and Table 10 shows that removing projection or non-keyframe context increases mean latent error and reduces PSNR.

4.5 User Study

We conducted a 82-participant user study for human preference evaluation. The user study results are summarised in Table 12. For each comparison, p-values were computed using binomial tests after excluding ties. For the comparisons against SVG and SAP, we used one-sided tests (H_1: Ours > Opponent); for the comparison against Dense, a two-sided test (H_0: no difference) was applied. Our method outperformed SVG in all three metrics, with statistically significant differences. For visual quality, 51.2% of participants preferred ours versus 30.5% for SVG. Temporal coherence favoured ours by 53.7% to 20.7%. Our method achieved even larger margins over SAP, where all differences are highly significant. Against Dense, the results are not statistically significant. The two-sided tests indicate no evidence of a difference between our method and Dense across all metrics.

5 Conclusion

In this paper, we present **RhymeFlow**, a training-free acceleration framework that revisits the computational paradigm of video diffusion models. Instead of synchronously updating all frames at every denoising step, RhymeFlow introduces a content-aware asynchronous schedule that differentiates semantically important keyframes from predictable non-keyframes. By assigning heterogeneous update frequencies and approximated trajectories to different frame groups, our method substantially reduces redundant computation while maintaining high visual fidelity.

Beyond its immediate efficiency gains, RhymeFlow reveals several promising directions for future exploration. One avenue is to replace the manually designed progressive schedule with learned, continuous scheduling functions that adapt to scene dynamics and model behavior. Moreover, the framework is inherently orthogonal to existing acceleration techniques such as sparse attention, quantization, and token pruning, enabling compounded improvements when combined. By shifting the focus from uniform denoising to intelligent, content-aware scheduling, RhymeFlow offers a principled path toward scalable and accessible high-resolution video generation.

Acknowledgements. This work was supported in part by the Beijing Natural Science Foundation of China under Grant L252011, by the National Natural Science Foundation of China under Grant 62576185, and by the Young Elite Scientist Sponsorship Program by CAST under Grant YESS20240544.

References

[1] Anurag Arnab, Mostafa Dehghani, Georg Heigold, Chen Sun, Mario Lučić, and Cordelia Schmid. Vivit: A video vision transformer. In Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), pages 6836–6846, October 2021.

[2] Daniel Bolya and Judy Hoffman. Token merging for fast stable diffusion. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pages 4599–4603, 2023.

[3] Tim Brooks, Bill Peebles, Connor Holmes, Will DePue, Yufei Guo, Li Jing, David Schnurr, Joe Taylor, Troy Luhman, Eric Luhman, Clarence Ng, Ricky Wang, and Aditya Ramesh. Video generation models as world simulators. 2024.

[4] Jiazi Bu, Pengyang Ling, Yujie Zhou, Yibin Wang, Yuhang Zang, Dahua Lin, and Jiaqi Wang. Dicache: Let diffusion model determine its own cache. arXiv preprint arXiv:2508.17356, 2025.

[5] Haoxin Chen, Yong Zhang, Xiaodong Cun, Menghan Xia, Xintao Wang, Chao Weng, and Ying Shan. Videocrafter2: Overcoming data limitations for high-quality video diffusion models. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pages 7310–7320, 2024.

[6] Jingxi Chen, Zongxia Li, Zhichao Liu, Guangyao Shi, Xiyang Wu, Fuxiao Liu, Cornelia Fermuller, Brandon Y Feng, and Yiannis Aloimonos. First frame is the place to go for video content customization. arXiv preprint arXiv:2511.15700, 2025.

[7] Florinel-Alin Croitoru, Vlad Hondru, Radu Tudor Ionescu, and Mubarak Shah. Diffusion models in vision: A survey. IEEE transactions on pattern analysis and machine intelligence, 45(9):10850–10869, 2023.

[8] Haoge Deng, Ting Pan, Haiwen Diao, Zhengxiong Luo, Yufeng Cui, Huchuan Lu, Shiguang Shan, Yonggang Qi, and Xinlong Wang. Autoregressive video generation without vector quantization. arXiv preprint arXiv:2412.14169, 2024.

[9] Rohit Girdhar, Mannat Singh, Andrew Brown, Quentin Duval, Samaneh Azadi, Sai Saketh Rambhatla, Akbar Shah, Xi Yin, Devi Parikh, and Ishan Misra. Factorizing text-to-video generation by explicit image conditioning. In European Conference on Computer Vision, pages 205–224. Springer, 2024.

[10] Agrim Gupta, Lijun Yu, Kihyuk Sohn, Xiuye Gu, Meera Hahn, Fei-Fei Li, Irfan Essa, Lu Jiang, and José Lezama. Photorealistic video generation with diffusion models. In European Conference on Computer Vision, pages 393–411. Springer, 2024.

[11] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin, editors, Advances in Neural Information Processing Systems, volume 33, pages 6840–6851. Curran Associates, Inc., 2020.

[12] Jonathan Ho, Tim Salimans, Alexey Gritsenko, William Chan, Mohammad Norouzi, and David J Fleet. Video diffusion models. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, Advances in Neural Information Processing Systems, volume 35, pages 8633–8646. Curran Associates, Inc., 2022.

[13] Wenyi Hong, Ming Ding, Wendi Zheng, Xinghan Liu, and Jie Tang. Cogvideo: Large-scale pretraining for text-to-video generation via transformers. arXiv preprint arXiv:2205.15868, 2022.

[14] Ziqi Huang, Yinan He, Jiashuo Yu, Fan Zhang, Chenyang Si, Yuming Jiang, Yuanhan Zhang, Tianxing Wu, Qingyang Jin, Nattapol Chanpaisit, Yaohui Wang, Xinyuan Chen, Limin Wang, Dahua Lin, Yu Qiao, and Ziwei Liu. VBench: Comprehensive benchmark suite for video generative models. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2024.

[15] Huiqiang Jiang, Yucheng Li, Chengruidong Zhang, Qianhui Wu, Xufang Luo, Surin Ahn, Zhenhua Han, Amir H Abdi, Dongsheng Li, Chin-Yew Lin, et al. Minference 1.0: Accelerating pre-filling for long-context llms via dynamic sparse attention. Advances in Neural Information Processing Systems, 37:52481–52515, 2024.

[16] Kumara Kahatapitiya, Haozhe Liu, Sen He, Ding Liu, Menglin Jia, Chenyang Zhang, Michael S Ryoo, and Tian Xie. Adaptive caching for faster video generation with diffusion transformers. In Proceedings of the IEEE/CVF International Conference on Computer Vision, pages 15240–15252, 2025.

[17] Levon Khachatryan, Andranik Movsisyan, Vahram Tadevosyan, Roberto Henschel, Zhangyang Wang, Shant Navasardyan, and Humphrey Shi. Text2video-zero: Text-to-image diffusion models are zero-shot video generators. In Proceedings of the IEEE/CVF International Conference on Computer Vision, pages 15954–15964, 2023.

[18] Weijie Kong, Qi Tian, Zijian Zhang, Rox Min, Zuozhuo Dai, Jin Zhou, Jiangfeng Xiong, Xin Li, Bo Wu, Jianwei Zhang, et al. Hunyuanvideo: A systematic framework for large video generative models. arXiv preprint arXiv:2412.03603, 2024.

[19] Chengxuan Li, Di Huang, Zeyu Lu, Yang Xiao, Qingqi Pei, and Lei Bai. A survey on long video generation: Challenges, methods, and prospects. arXiv preprint arXiv:2403.16407, 2024.

[20] Muyang Li*, Yujun Lin*, Zhekai Zhang*, Tianle Cai, Xiuyu Li, Junxian Guo, Enze Xie, Chenlin Meng, Jun-Yan Zhu, and Song Han. Svdquant: Absorbing outliers by low-rank components for 4-bit diffusion models. In The Thirteenth International Conference on Learning Representations, 2025.

[21] Xiuyu Li, Yijiang Liu, Long Lian, Huanrui Yang, Zhen Dong, Daniel Kang, Shanghang Zhang, and Kurt Keutzer. Q-diffusion: Quantizing diffusion models. In Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), pages 17535–17545, October 2023.

[22] Cheng Lu, Yuhao Zhou, Fan Bao, Jianfei Chen, Chongxuan LI, and Jun Zhu. Dpm-solver: A fast ode solver for diffusion probabilistic model sampling in around 10 steps. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, Advances in Neural Information Processing Systems, volume 35, pages 5775–5787. Curran Associates, Inc., 2022.

[23] Cheng Lu, Yuhao Zhou, Fan Bao, Jianfei Chen, Chongxuan Li, and Jun Zhu. Dpm-solver++: Fast solver for guided sampling of diffusion probabilistic models. Machine Intelligence Research, 22(4):730–751, June 2025.

[24] Yu Lu, Yuanzhi Liang, Linchao Zhu, and Yi Yang. Freelong: Training-free long video generation with spectralblend temporal attention. Advances in Neural Information Processing Systems, 37:131434–131455, 2024.

[25] Zhengyao Lv, Chenyang Si, Junhao Song, Zhenyu Yang, Yu Qiao, Ziwei Liu, and Kwan-Yee K. Wong. Fastercache: Training-free video diffusion model acceleration with high quality. In arxiv, 2024.

[26] Xin Ma, Yaohui Wang, Gengyun Jia, Xinyuan Chen, Ziwei Liu, Yuan-Fang Li, Cunjian Chen, and Yu Qiao. Latte: Latent diffusion transformer for video generation. arXiv preprint arXiv:2401.03048, 2024.

[27] Xinyin Ma, Gongfan Fang, and Xinchao Wang. Deepcache: Accelerating diffusion models for free, 2023.

[28] William Peebles and Saining Xie. Scalable diffusion models with transformers. In Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), pages 4195–4205, October 2023.

[29] Weiming Ren, Huan Yang, Ge Zhang, Cong Wei, Xinrun Du, Wenhao Huang, and Wenhu Chen. Consisti2v: Enhancing visual consistency for image-to-video generation. arXiv preprint arXiv:2402.04324, 2024.

[30] Tim Salimans and Jonathan Ho. Progressive distillation for fast sampling of diffusion models. In International Conference on Learning Representations, 2022.

[31] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In International Conference on Learning Representations, 2021.

[32] Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models. arXiv preprint arXiv:2303.01469, 2023.

[33] Yoad Tewel, Omri Kaduri, Rinon Gal, Yoni Kasten, Lior Wolf, Gal Chechik, and Yuval Atzmon. Training-free consistent text-to-image generation. ACM Transactions on Graphics (TOG), 43(4):1–18, 2024.

[34] Team Wan, Ang Wang, Baole Ai, Bin Wen, Chaojie Mao, Chen-Wei Xie, Di Chen, Feiwu Yu, Haiming Zhao, Jianxiao Yang, Jianyuan Zeng, Jiayu Wang, Jingfeng Zhang, Jingren Zhou, Jinkai Wang, Jixuan Chen, Kai Zhu, Kang Zhao, Keyu Yan, Lianghua Huang, Mengyang Feng, Ningyi Zhang, Pandeng Li, Pingyu Wu, Ruihang Chu, Ruili Feng, Shiwei Zhang, Siyang Sun, Tao Fang, Tianxing Wang, Tianyi Gui, Tingyu Weng, Tong Shen, Wei Lin, Wei Wang, Wei Wang, Wenmeng Zhou, Wente Wang, Wenting Shen, Wenyuan Yu, Xianzhong Shi, Xiaoming Huang, Xin Xu, Yan Kou, Yangyu Lv, Yifei Li, Yijing Liu, Yiming Wang, Yingya Zhang, Yitong Huang, Yong Li, You Wu, Yu Liu, Yulin Pan, Yun Zheng, Yuntao Hong, Yupeng Shi, Yutong Feng, Zeyinzi Jiang, Zhen Han, Zhi-Fan Wu, and Ziyu Liu. Wan: Open and advanced large-scale video generative models. arXiv preprint arXiv:2503.20314, 2025.

[35] Johannes G Wijmans and Richard W Baker. The solution-diffusion model: a review. Journal of membrane science, 107(1-2):1–21, 1995.

[36] Haocheng Xi, Shuo Yang, Yilong Zhao, Chenfeng Xu, Muyang Li, Xiuyu Li, Yujun Lin, Han Cai, Jintao Zhang, Dacheng Li, et al. Sparse videogen: Accelerating video diffusion transformers with spatial-temporal sparsity. arXiv preprint arXiv:2502.01776, 2025.

[37] Yifei Xia, Suhan Ling, Fangcheng Fu, Yujie Wang, Huixia Li, Xuefeng Xiao, and Bin Cui. Training-free and adaptive sparse attention for efficient long video generation. arXiv preprint arXiv:2502.21079, 2025.

[38] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. arXiv preprint arXiv:2309.17453, 2023.

[39] Zhen Xing, Qijun Feng, Haoran Chen, Qi Dai, Han Hu, Hang Xu, Zuxuan Wu, and Yu-Gang Jiang. A survey on video diffusion models. ACM Computing Surveys, 57(2):1–42, 2024.

[40] Shuo Yang, Haocheng Xi, Yilong Zhao, Muyang Li, Jintao Zhang, Han Cai, Yujun Lin, Xiuyu Li, Chenfeng Xu, Kelly Peng, et al. Sparse videogen2: Accelerate video generation with sparse attention via semantic-aware permutation. arXiv preprint arXiv:2505.18875, 2025.

[41] Zhuoyi Yang, Jiayan Teng, Wendi Zheng, Ming Ding, Shiyu Huang, Jiazheng Xu, Yuanming Yang, Wenyi Hong, Xiaohan Zhang, Guanyu Feng, et al. Cogvideox: Text-to-video diffusion models with an expert transformer. arXiv preprint arXiv:2408.06072, 2024.

[42] Jiwen Yu, Yinhuai Wang, Chen Zhao, Bernard Ghanem, and Jian Zhang. Freedom: Training-free energy-guided conditional diffusion model. In Proceedings of the IEEE/CVF International Conference on Computer Vision, pages 23174–23184, 2023.

[43] Zhihang Yuan, Rui Xie, Yuzhang Shang, Hanling Zhang, Siyuan Wang, Shengen Yan, Guohao Dai, and Yu Wang. Vgdfr: Diffusion-based video generation with dynamic latent frame rate, 2025.

[44] Evelyn Zhang, Jiayi Tang, Xuefei Ning, and Linfeng Zhang. Training-free and hardware-friendly acceleration for diffusion models via similarity-based token pruning. In Proceedings of the Thirty-Ninth AAAI Conference on Artificial Intelligence and Thirty-Seventh Conference on Innovative Applications of Artificial Intelligence and Fifteenth Symposium on Educational Advances in Artificial Intelligence, AAAI'25/IAAI'25/EAAI'25. AAAI Press, 2025.

[45] Jintao Zhang, Haofeng Huang, Pengle Zhang, Jia Wei, Jun Zhu, and Jianfei Chen. Sageattention2: Efficient attention with thorough outlier smoothing and per-thread int4 quantization. In International Conference on Machine Learning (ICML), 2025.

[46] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang "Atlas" Wang, and Beidi Chen. H2o: Heavy-hitter oracle for efficient generative inference of large language models. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, Advances in Neural Information Processing Systems, volume 36, pages 34661–34710. Curran Associates, Inc., 2023.

[47] Xuanlei Zhao, Xiaolong Jin, Kai Wang, and Yang You. Real-time video generation with pyramid attention broadcast. arXiv preprint arXiv:2408.12588, 2024.

[48] Kaiwen Zheng, Cheng Lu, Jianfei Chen, and Jun Zhu. Dpm-solver-v3: Improved diffusion ode solver with empirical model statistics. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, Advances in Neural Information Processing Systems, volume 36, pages 55502–55542. Curran Associates, Inc., 2023.

[49] Xin Zhou, Dingkang Liang, Kaijin Chen, Tianrui Feng, Xiwu Chen, Hongkai Lin, Yikang Ding, Feiyang Tan, Hengshuang Zhao, and Xiang Bai. Less is enough: Training-free video diffusion acceleration via runtime-adaptive caching. arXiv preprint arXiv:2507.02860, 2025.

A More Experimental Results

A.1 Additional Visualization Results

We present further qualitative visual comparisons in Figure 5. While image sequences offer a static perspective, the temporal coherence and motion dynamics are best observed in motion. Therefore, we encourage readers to review the extended video samples provided in the accompanying multimedia materials for a more comprehensive visual assessment of our approach.

Figure 5 Additional qualitative results on Wan 2.1. We compare the sequences generated by the original model (utilizing full dense attention) against those produced by our RhymeFlow. Our method successfully preserves intricate details and complex motion dynamics without introducing noticeable artifacts.

A.2 Evaluation on Long-Duration Videos

While the main paper primarily evaluates models under the standard 81-frame context, we further extend our experiments to long-duration (240 frames) videos to rigorously assess the scalability and robustness of our framework.

As demonstrated in Table 9, RhymeFlow exhibits remarkable stability across various parameter configurations

Table 9 Parameter sensitivity analysis on long-duration videos. We ablate the warm-up duration (T_w) and the keyframe budget (M) on Wan 2.1.

Method	PSNR ↑	SSIM ↑	LPIPS ↓	Latency (s) ↓	Speedup ↑
Original	-	-	-	5599	-
Ours ($T_w = 10, M = 15$)	25.122	0.806	0.251	3034	1.85 ×
Ours ($T_w = 10, M = 21$)	25.169	0.809	0.247	3485	1.61 ×
Ours ($T_w = 10, M = 30$)	25.339	0.818	0.241	3959	1.41 ×
Ours ($T_w = 15, M = 15$)	27.011	0.813	0.232	3467	1.61 ×
Ours ($T_w = 15, M = 21$)	**27.256**	**0.822**	**0.224**	**3626**	**1.54** ×
Ours ($T_w = 15, M = 30$)	27.832	0.840	0.210	4164	1.34 ×
Ours ($T_w = 20, M = 15$)	27.585	0.857	0.205	3605	1.55 ×
Ours ($T_w = 20, M = 21$)	27.597	0.858	0.203	3908	1.43 ×
Ours ($T_w = 20, M = 30$)	28.009	0.863	0.198	4369	1.28 ×

Table 10 Quantitative comparison against SOTA baseline methods on long-duration videos. RhymeFlow achieves the optimal balance of efficiency and visual quality.

Method	PSNR ↑	SSIM ↑	LPIPS ↓	Latency (s) ↓	Speedup ↑
Original	-	-	-	5599	-
SVG [36]	21.139	0.705	0.344	3943	1.42 ×
SAP [40]	26.050	0.836	0.217	2978	1.88 ×
Ours	27.256	0.822	0.224	3626	1.54 ×
Ours + SVG	**26.585**	**0.842**	**0.211**	**2931**	**1.91** ×

(T_w and M), maintaining high structural similarity and perceptual quality. Furthermore, we benchmarked RhymeFlow against state-of-the-art (SOTA) training-free acceleration baselines on these extended video sequences. Detailed in Table 10, our method and its synergistic variant (Ours+SVG) consistently achieve a superior trade-off, delivering up to 1.91× acceleration while preserving high visual fidelity compared to existing masking-based counterparts.

A.3 Ablation Study on Progressive Scheduling Parameters

As detailed in the main paper, our progressive asynchronous scheduling mechanism relies on the update stride parameters (n_{small} and n_{large}). We perform an ablation study on these settings using Wan 2.1, with the results summarized in Table 11. The findings indicate a clear empirical trade-off: larger strides prioritize inference speed over generation fidelity. This validates our default configuration ($n_{\text{small}} = 2, n_{\text{large}} = 3$) as the optimal balance for ensuring high visual quality while delivering substantial computational acceleration.

A.4 User Study

To rigorously validate the perceptual fidelity of RhymeFlow from a human-centric perspective, we conducted a double-blind A/B test. Twenty participants evaluated the output videos conditioned on 20 diverse and challenging prompts. We benchmarked RhymeFlow against two SOTA acceleration baselines (SVG [36] and SAP [40]), as well as the original model (Wan 2.1).

The results, presented in Table 12, demonstrate that RhymeFlow significantly outperforms intra-step sparse-attention baselines. Notably, in terms of Temporal Coherence, our method secured a 54.2% win rate against SVG, confirming that our Latent Trajectory Projection effectively mitigates the motion jitter and incoherence commonly caused by standard masking-based methods. Crucially, when compared against the Original model, evaluators overwhelmingly reported a "Tie" across all metrics (ranging from 62.5% to 66.6%), underscoring

Table 11 Ablation study on the stride parameters of the progressive scheduling ($T_w = 15, M = 7$).

Method	PSNR ↑	SSIM ↑	LPIPS ↓	Latency (s) ↓	Speedup ↑
Original	-	-	-	960	-
Ours ($n_{\text{small}} = 2, n_{\text{large}} = 3$)	**27.287**	**0.814**	**0.210**	**627**	**1.53** ×
Ours ($n_{\text{small}} = 2, n_{\text{large}} = 4$)	25.178	0.804	0.251	596	1.61 ×
Ours ($n_{\text{small}} = 2, n_{\text{large}} = 5$)	24.459	0.787	0.268	568	1.69 ×
Ours ($n_{\text{small}} = 3, n_{\text{large}} = 4$)	23.546	0.739	0.294	604	1.59 ×
Ours ($n_{\text{small}} = 3, n_{\text{large}} = 5$)	21.995	0.724	0.322	561	1.71 ×
Ours ($n_{\text{small}} = 3, n_{\text{large}} = 6$)	21.125	0.685	0.339	530	1.81 ×

Table 12 Results of the double-blind user study. Values indicate the percentage of user preferences across pairwise comparisons.

Comparison	Evaluation Metric	Ours Preferred	Tie	Competitor Preferred
Ours vs. SVG	Visual Quality	**45.8%**	37.5%	16.7%
	Temporal Coherence	**54.2%**	33.3%	12.5%
	Perceptual Preference	**50.0%**	37.5%	12.5%
Ours vs. SAP	Visual Quality	37.5%	**45.8%**	16.7%
	Temporal Coherence	**45.9%**	33.3%	20.8%
	Perceptual Preference	**45.8%**	41.7%	12.5%
Ours vs. Original	Visual Quality	12.5%	**62.5%**	25.0%
	Temporal Coherence	16.7%	**66.6%**	16.7%
	Perceptual Preference	12.5%	**62.5%**	25.0%

that RhymeFlow delivers a 1.53× speedup with negligible perceptual degradation to human eyes.

B More Theoretical Analysis

B.1 Theoretical Analysis on FLOPs

In the main paper, we evaluate our acceleration effects primarily through experimental time consumption results. In this section, we provide a theoretical analysis of the speed-up ratio from the perspective of Floating Point Operations (FLOPs). Let us revisit our pipeline through the lens of FLOPs analysis.

Model Parameters:

- L: Number of transformer layers
- H: Number of attention heads per layer
- d: Dimension of each attention head
- $D = H \times d$: Total hidden dimension

Video Parameters:

- F: Number of video frames (after VAE temporal compression)
- N: Number of tokens per frame (i.e., spatial tokens)
- $S = F \times N$: Total sequence length per sample
- C: Context length (text prompt tokens, $C = 0$ for Wan)

Denoising Parameters:

- T: Total number of denoising steps
- T_{warmup}: Number of warmup steps with dense attention
- M: Number of identified keyframes ($M \ll F$)
- n: Skip interval for non-keyframes ($n \geq 2$)
- $T_{\text{RhymeFlow}} = T - T_{\text{warmup}}$: Number of RhymeFlow phase steps

Specifically, we use the Wan2.1-T2V-1.3B model configuration throughout our analysis. We now concentrate on the basic asynchronous scheduling implementations (without progressive scheduling), detailed statistics are illustrated in Table 13.

Table 13 Wan2.1 Model Configuration.

Parameter	Value
Transformer layers (L)	30
Attention heads (H)	12
Head dimension (d)	128
Hidden dimension (D)	1,536
Video resolution	720p (1280×720)
Original frames	81
Compressed frames (F)	21
Tokens per frame (N)	3,600
Total tokens (S)	75,600
Total steps (T)	50
Warmup steps (T_{warmup})	10
Keyframes (M)	5
Skip interval (n)	2

B.1.1 Dense Baseline FLOPS Analysis.

We first analyze the computational cost of the standard dense attention mechanism, which serves as our baseline. A single self-attention layer at one denoising timestep involves the following operations:

QKV Projection. Transform input hidden states into query, key, and value representations:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V, \tag{4}$$

where $\mathbf{X} \in \mathbb{R}^{S \times D}$ and $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{D \times D}$. The total FLOPs for QKV projection:

$$\text{FLOPs}_{\text{QKV}} = 3 \times (S \times D \times D) = 3SD^2. \tag{5}$$

Attention Score Computation. Compute pairwise attention scores $\mathbf{A} = \mathbf{Q}\mathbf{K}^\top / \sqrt{d}$:

$$\text{FLOPs}_{\text{scores}} = S \times S \times D = S^2 D. \tag{6}$$

Softmax Normalization. Apply softmax to obtain attention weights. While softmax involves exponentials and normalization, for FLOP counting we approximate this as:

$$\text{FLOPs}_{\text{softmax}} \approx 3S^2, \tag{7}$$

which is typically negligible compared to matrix multiplications for large D.

Attention-Weighted Aggregation. Compute output as $\mathbf{O} = \text{softmax}(\mathbf{A})\mathbf{V}$:

$$\text{FLOPs}_{\text{aggregate}} = S \times S \times D = S^2 D. \tag{8}$$

Output Projection. Project concatenated multi-head outputs back to hidden dimension:

$$\text{FLOPs}_{\text{out}} = S \times D \times D = SD^2. \tag{9}$$

Summing all components:

$$\begin{aligned} \text{FLOPs}_{\text{layer}} &= 3SD^2 + S^2D + 3S^2 + S^2D + SD^2 \\ &= 4SD^2 + 2S^2D + 3S^2. \end{aligned} \tag{10}$$

For large-scale models where S and D are both large, the quadratic attention term $2S^2D$ typically dominates. We can approximate:

$$\text{FLOPs}_{\text{layer}} \approx 4SD^2 + 2S^2D = 2D(2SD + S^2). \tag{11}$$

For the complete dense denoising process with L layers and T timesteps:

$$\text{FLOPs}_{\text{dense}} = L \times T \times \text{FLOPs}_{\text{layer}} = L \times T \times 2D(2SD + S^2). \tag{12}$$

Using the configuration in Table 13:

$$\begin{aligned} \text{FLOPs}_{\text{layer}} &= 4 \times 75{,}600 \times 1{,}536^2 + 2 \times 75{,}600^2 \times 1{,}536 \\ &= 713{,}666{,}227{,}200 + 17{,}559{,}660{,}544{,}000 \\ &\approx 18.27 \times 10^{12} \text{ FLOPs} = 18.27 \text{ TFLOPs}. \end{aligned} \tag{13}$$

The attention core (quadratic term) accounts for:

$$\frac{2S^2D}{\text{FLOPs}_{\text{layer}}} = \frac{17.56}{18.27} \approx 96.1\%, \tag{14}$$

confirming that attention dominates the computation. Total dense baseline FLOPs:

$$\begin{aligned} \text{FLOPs}_{\text{dense}} &= 30 \times 50 \times 18.27 = 27{,}405 \text{ TFLOPs} \\ &= 27.41 \text{ PFLOPs}. \end{aligned} \tag{15}$$

B.1.2 RhymeFlow FLOPS Analysis.

We now analyze the computational cost of our Selective Step Skipping method, which consists of two phases: warmup and selective denoising.

Warm up Stage. During the warmup phase ($t \le T_{\text{warmup}}$), all F frames perform dense attention at every step to establish initial distributions. The FLOPs are identical to the dense baseline:

$$\text{FLOPs}_{\text{warmup}} = L \times T_{\text{warmup}} \times 2D(2SD + S^2). \tag{16}$$

After warmup, we partition frames into keyframes $\mathcal{F}_k$ (with $|\mathcal{F}_k| = M$) and non-keyframes $\mathcal{F}_n$ (with $|\mathcal{F}_n| = F - M$). The denoising schedule is:

- Keyframes ($f \in \mathcal{F}_k$): Denoise at every step
- Non-keyframes ($f \in \mathcal{F}_n$): Denoise only when $(t - T_{\text{warmup}}) \bmod n = 0$

Skip Steps. At timesteps where $(t - T_{\text{warmup}}) \bmod n \neq 0$, only keyframes compute attention outputs (non-keyframes use interpolated representations). The effective sequence length is:

$$S_{\text{skip}} = M \times N. \tag{17}$$

Number of skip steps in RhymeFlow phase:

$$N_{\text{skip}} = \left\lfloor T_{\text{RhymeFlow}} \times \frac{n-1}{n} \right\rfloor. \tag{18}$$

FLOPs for skip steps:

$$\begin{aligned}\text{FLOPs}_{\text{skip}} &= L \times N_{\text{skip}} \times 2D(2S_{\text{skip}}D + S_{\text{skip}}^2) \\ &= L \times N_{\text{skip}} \times 2D(2MND + M^2N^2).\end{aligned} \tag{19}$$

Full Denoising Steps. At timesteps where $(t - T_{\text{warmup}}) \bmod n = 0$, both keyframes and non-keyframes denoise. All F frames participate:

$$S_{\text{full}} = F \times N = S. \tag{20}$$

Number of full denoising steps:

$$N_{\text{denoise}} = T_{\text{RhymeFlow}} - N_{\text{skip}} \approx \frac{T_{\text{RhymeFlow}}}{n}. \tag{21}$$

FLOPs for full denoising steps:

$$\text{FLOPs}_{\text{denoise}} = L \times N_{\text{denoise}} \times 2D(2FND + F^2N^2). \tag{22}$$

Combining both phases:

$$\text{FLOPs}_{\text{RhymeFlow}} = \text{FLOPs}_{\text{warmup}} + \text{FLOPs}_{\text{skip}} + \text{FLOPs}_{\text{denoise}}. \tag{23}$$

Substituting Equations (16), (19), and (22):

$$\begin{aligned}\text{FLOPs}_{\text{RhymeFlow}} = {} & L \times T_{\text{warmup}} \times 2D(2FND + F^2N^2) \\ & + L \times N_{\text{skip}} \times 2D(2MND + M^2N^2) \\ & + L \times N_{\text{denoise}} \times 2D(2FND + F^2N^2).\end{aligned} \tag{24}$$

B.1.3 Numerical Example: Wan 2.1

Using $T = 50$, $T_{\text{warmup}} = 10$, $M = 5$, $n = 2$, $F = 21$:

$$T_{\text{RhymeFlow}} = 50 - 10 = 40, \tag{25}$$

$$N_{\text{skip}} = \left\lfloor 40 \times \frac{1}{2} \right\rfloor = 20, \tag{26}$$

$$N_{\text{denoise}} = 40 - 20 = 20. \tag{27}$$

Warmup FLOPs.

$$\begin{aligned}\text{FLOPs}_{\text{warmup}} &= 30 \times 10 \times 18.27 = 5{,}481 \text{ TFLOPs} \\ &= 5.48 \text{ PFLOPs}.\end{aligned} \tag{28}$$

Skip Steps FLOPs. First compute $\text{FLOPs}_{\text{layer, skip}}$ for sequence length $S_{\text{skip}} = 5 \times 3{,}600 = 18{,}000$:

$$\begin{aligned}\text{FLOPs}_{\text{layer, skip}} &= 4 \times 18{,}000 \times 1{,}536^2 + 2 \times 18{,}000^2 \times 1{,}536 \\ &= 170{,}074{,}521{,}600 + 1{,}990{,}656{,}000{,}000 \\ &\approx 2.16 \times 10^{12} \text{ FLOPs} = 2.16 \text{ TFLOPs}.\end{aligned} \tag{29}$$

Total skip FLOPs:

$$\text{FLOPs}_{\text{skip}} = 30 \times 20 \times 2.16 = 1{,}296 \text{ TFLOPs} = 1.30 \text{ PFLOPs}. \tag{30}$$

Denoise Steps FLOPs.

$$\begin{aligned}\text{FLOPs}_{\text{denoise}} &= 30 \times 20 \times 18.27 = 10{,}962 \text{ TFLOPs} \\ &= 10.96 \text{ PFLOPs}.\end{aligned} \tag{31}$$

Total RhymeFlow FLOPs.

$$\text{FLOPs}_{\text{RhymeFlow}} = 5.48 + 1.30 + 10.96 = 17.74 \text{ PFLOPs}. \tag{32}$$

B.1.4 Theoretical Speedup Analysis

The theoretical speedup from FLOP reduction:

$$\text{Speedup}_{\text{FLOPs}} = \frac{\text{FLOPs}_{\text{dense}}}{\text{FLOPs}_{\text{RhymeFlow}}}. \tag{33}$$

For Wan 2.1:

$$\text{Speedup}_{\text{FLOPs}} = \frac{27.41}{17.74} = 1.545\times. \tag{34}$$

This corresponds to:

$$\text{FLOP Reduction} = 1 - \frac{17.74}{27.41} = 35.3\%. \tag{35}$$

The reported speedup ratio of 1.53× is slightly lower than the theoretical speedup of 1.545×. This discrepancy results from the non-ideal scaling of hardware efficiency and algorithmic overheads not captured by pure FLOPs counting. We attribute this gap to two primary factors:

- **Algorithmic Overheads:** The theoretical analysis assumes zero cost for control logic. However, the practical implementation of RhymeFlow introduces necessary computations:
  1. Keyframe Identification: The computation of frame-to-frame latent similarity (e.g., cosine similarity) and the selection algorithm (clustering or thresholding) consume GPU cycles.
  2. Latent Trajectories Projection: Generating intermediate states (x_{t-1}) for skipped frames via flow-based latent projection requires additional vector operations, which, while lightweight, are not negligible.
- **Hardware Efficiency & Memory Access Constraints:** The reduction in FLOPs does not translate linearly to latency reduction due to decreased GPU utilization during the skip steps.
  1. Reduced Parallelism: During skip steps, the model processes only $M = 5$ keyframes instead of the full $F = 21$ frames. This reduces the attention sequence length from $S_{\text{full}} = 75,600$ to $S_{\text{skip}} = 18,000$.
  2. GPU Occupancy Drop: On high-performance GPUs, such a significant reduction in sequence length ($\sim 76\%$ decrease) lowers the kernel occupancy. The workload shifts from being compute-bound to memory-bound, meaning the GPU cores spend more time waiting for data transfer than performing calculations. Consequently, the effective TFLOPs/s achieved during skip steps is lower than during dense full-sequence processing.

Therefore, the measured speedup of 1.53× represents a robust trade-off between theoretical reduction and hardware utilization efficiency.

B.2 Theoretical Analysis on KV-Caching

In the main paper, we introduce our KV-Caching mechanism in a simple manner. Now let's dive into the design of per-layer rolling-cache and analyze how efficient it is compared to the original KV management. A key challenge in selective step skipping is maintaining temporal coherence for non-keyframes that do not perform denoising at certain timesteps. Traditional KV-caching mechanisms, widely adopted in autoregressive language models, exploit causal dependencies to reuse previously computed key-value pairs. However, these approaches are fundamentally inapplicable to video diffusion models due to three critical distinctions: (1) non-autoregressive generation, where all frames are updated simultaneously at each denoising step; (2) bidirectional temporal dependencies, requiring full attention across all frames without causal constraints; and (3) state evolution, where the latent representations of all tokens change at every timestep.

To address this challenge, we propose a **rolling KV-cache strategy** that enables latent projection of intermediate frame states during skipped denoising steps. Our approach maintains temporal consistency while achieving substantial memory efficiency compared to naive caching of all intermediate states.

B.2.1 Problem Formulation:

Let $\mathbf{z}_t^{(f)} \in \mathbb{R}^{N\times D}$ denote the latent representation of frame f at denoising timestep t, where N is the number of tokens per frame and D is the feature dimension. In the standard dense denoising schedule, every frame undergoes attention computation at every timestep:

$$\mathbf{z}_{t-1}^{(f)} = \text{Attention}\left(\mathbf{Q}_t^{(f)}, \mathbf{K}_t, \mathbf{V}_t\right) + \mathbf{z}_t^{(f)}, \tag{36}$$

where $\mathbf{K}_t, \mathbf{V}_t \in \mathbb{R}^{(F\cdot N)\times D}$ aggregate keys and values from all F frames.

In our selective step skipping regime, we partition frames into keyframes $\mathcal{F}_k$ and non-keyframes $\mathcal{F}_n$. At non-denoising steps, non-keyframes must still provide key-value representations for keyframe attention, but their query outputs need not be computed. The key challenge is how we can obtain $\mathbf{K}_t^{(f)}$ and $\mathbf{V}_t^{(f)}$ for $f \in \mathcal{F}_n$ at skipped timesteps without performing full attention computation?

B.2.2 Rolling Cache Design.

We introduce a **per-layer, per-frame rolling cache** that stores the attention outputs of the two most recent denoising timesteps for each non-keyframe. For frame $f \in \mathcal{F}_n$ at layer ℓ, the cache $\mathcal{C}_\ell^{(f)}$ maintains:

$$\mathcal{C}_\ell^{(f)} = \begin{Bmatrix} \mathbf{h}_{\text{before}}^{(\ell,f)} \in \mathbb{R}^{N\times D}, & t_{\text{before}}, \\ \mathbf{h}_{\text{after}}^{(\ell,f)} \in \mathbb{R}^{N\times D}, & t_{\text{after}}, \end{Bmatrix} \tag{37}$$

where $\mathbf{h}_{\text{before}}^{(\ell,f)}$ and $\mathbf{h}_{\text{after}}^{(\ell,f)}$ are the cached attention outputs at the two most recent denoising steps $t_{\text{before}} > t_{\text{after}}$ (noting that diffusion timesteps decrease during denoising). Crucially, we cache the **output hidden states** (post-attention) rather than input latents, as they already incorporate contextual information from all other frames.

At the end of the warmup phase (step t_{w}), we initialize the cache for all non-keyframes in all layers:

$$\begin{aligned} \mathbf{h}_{\text{before}}^{(\ell,f)} = \mathbf{h}_{\text{after}}^{(\ell,f)} = \text{Attention}^{(\ell)}\left(\mathbf{Q}_{t_{\text{warmup}}}^{(f)}, \mathbf{K}_{t_{\text{warmup}}}, \mathbf{V}_{t_{\text{warmup}}}\right), \\ t_{\text{before}} = t_{\text{after}} = t_{\text{w}}. \end{aligned} \tag{38}$$

At a skipped timestep t_{skip} where $t_{\text{after}} < t_{\text{skip}} < t_{\text{before}}$, we reconstruct the frame representation via latent trajectories projection:

$$\mathbf{h}_{t_{\text{skip}}}^{(\ell,f)} = (1-\alpha)\cdot \mathbf{h}_{\text{before}}^{(\ell,f)} + \alpha \cdot \mathbf{h}_{\text{after}}^{(\ell,f)}, \tag{39}$$

where the interpolation weight $\alpha \in [0,1]$ is computed based on the relative temporal position:

$$\alpha = \frac{t_{\text{before}} - t_{\text{skip}}}{t_{\text{before}} - t_{\text{after}} + \epsilon}, \quad \epsilon = 10^{-8}. \tag{40}$$

This formulation ensures that $\alpha \to 0$ as $t_{\text{skip}} \to t_{\text{before}}$ (favoring the earlier cached state) and $\alpha \to 1$ as $t_{\text{skip}} \to t_{\text{after}}$ (favoring the later cached state), aligning with the decreasing-timestep denoising trajectory.

Attention computation at skipped steps: While non-keyframes $f \in \mathcal{F}_n$ do not compute query outputs, they must still provide key-value pairs for keyframe attention. We directly use the latent projected hidden states:

$$\mathbf{K}_{t_{\text{skip}}}^{(f)} = \mathbf{V}_{t_{\text{skip}}}^{(f)} = \mathbf{h}_{t_{\text{skip}}}^{(\ell,f)}, \quad f \in \mathcal{F}_n. \tag{41}$$

Keyframes $f \in \mathcal{F}_k$ perform full attention over all frames:

$$\begin{aligned} \mathbf{h}_{t_{\text{skip}}}^{(\ell,f)} = \text{Attention}^{(\ell)}(\mathbf{Q}_{t_{\text{skip}}}^{(f)}, \left[\mathbf{K}_{t_{\text{skip}}}^{(1)}, \ldots, \mathbf{K}_{t_{\text{skip}}}^{(F)}\right], \\ \left[\mathbf{V}_{t_{\text{skip}}}^{(1)}, \ldots, \mathbf{V}_{t_{\text{skip}}}^{(F)}\right]), \quad f \in \mathcal{F}_k. \end{aligned} \tag{42}$$

Cache Update at Denoising Steps: When non-keyframes perform denoising at "Rhythmic Point" t_{denoise}, we update the rolling cache via a shift-and-store strategy:

$$\begin{aligned} \mathbf{h}_{\text{before}}^{(\ell,f)} &\leftarrow \mathbf{h}_{\text{after}}^{(\ell,f)}, \\ t_{\text{before}} &\leftarrow t_{\text{after}}, \\ \mathbf{h}_{\text{after}}^{(\ell,f)} &\leftarrow \text{Attn}^{(\ell)}\big(\mathbf{Q}_{t_{\text{denoise}}}^{(f)}, \mathbf{K}_{t_{\text{denoise}}}, \mathbf{V}_{t_{\text{denoise}}}\big), \\ t_{\text{after}} &\leftarrow t_{\text{denoise}}. \end{aligned} \tag{43}$$

This rolling update ensures that the cache always retains the two most recent denoising states, enabling accurate projection for the next $(n-1)$ skipped steps.

B.2.3 Cross-Layer Consistency Guarantees.

A subtle but critical challenge in multi-layer architectures is ensuring **temporal consistency** across layers. Inconsistent cache states can cause error propagation, as layer $\ell+1$ depends on the output of layer ℓ. We enforce consistency through three mechanisms:

- **Shared global step counter**: A class-level variable is incremented only in layer 0 and shared across all layer instances, ensuring all layers agree on the current timestep.
- **Synchronized cache updates**: All layers update their caches simultaneously at denoising steps, preventing temporal misalignment.
- **Shared keyframe indices**: The set $\mathcal{F}_k$ is determined once at the end of warmup (layer 0) and frozen thereafter, ensuring all layers use the same denoising schedule.

Formally, let $\tau(\ell)$ denote the effective timestep used by layer ℓ. Consistency requires $\tau(\ell) = \tau(\ell') = t_{\text{current}}$ for all ℓ, ℓ', which our design guarantees.

C Detailed Implementation of RhymeFlow

To provide a clear procedural understanding of the proposed framework, we present the algorithmic details of RhymeFlow. The process is divided into two primary stages: (1) Content-aware sequential keyframe selection, and (2) Asynchronous denoising flow scheduling.

C.1 Sequential Keyframe Selection

As discussed in Section 3.1 of the main paper, traditional uniform frame selection fails to capture non-linear semantic transitions, while computing similarity on noisy latents $\mathbf{z}_T$ is unreliable due to noise interference. We first perform a single-step denoising proxy to estimate the clean latents $\hat{\mathbf{z}}_0$ as a robust basis for selection. To prevent keyframes from clustering too densely or being too sparse due to a fixed similarity threshold, we propose a Dynamic Threshold Adjustment mechanism, as detailed in Algorithm 1. We define an expected average interval $L_{\text{avg}} = N/M$. If the temporal gap between the current and previous keyframe significantly exceeds L_{avg} (controlled by δ_{up}), the threshold τ is decreased by $\Delta\tau$ to encourage selection; conversely, if the gap is too small (controlled by δ_{down}), τ is increased to suppress redundant keyframes. This ensures a content-adaptive yet well-distributed keyframe set.

C.2 The Overall RhymeFlow Pipeline

The core of RhymeFlow lies in its heterogeneous scheduling. While keyframes maintain a standard step-by-step denoising trajectory to preserve structural integrity, non-keyframes follow an accelerated path by skipping redundant steps. Crucially, during skipped steps, we employ Latent Trajectory Projection to synthesize the missing temporal context, ensuring that keyframe updates remain globally coherent. This process is summarized in Algorithm 2.

Algorithm 1 Dynamic Sequential Keyframe Selection

Require: Initial noisy latents $\{\mathbf{z}_T^{(i)}\}_{i=1}^N$, Initial threshold τ, Max keyframe budget M, Threshold step $\Delta\tau$, Interval tolerance $\{\delta_{up}, \delta_{\text{down}}\}$.
Ensure: Keyframe set $\mathcal{K}$, Non-keyframe set $\mathcal{N}$.

// Step 1: Predict clean latent proxies
$\{\hat{\mathbf{z}}_0^{(i)}\}_{i=1}^N \leftarrow \text{Denoise}(\{\mathbf{z}_T^{(i)}\}, T \to 0)$ ▷ Single-step approximation
// Step 2: Adaptive sequential selection
$\mathcal{K} \leftarrow \{1\}$, $idx_{\text{last}} \leftarrow 1$, $L_{\text{avg}} \leftarrow N/M$
for $i = 2$ **to** N **do**
  if $|\mathcal{K}| < M$ **then**
    $S \leftarrow \text{CosineSimilarity}(\hat{\mathbf{z}}_0^{(i)}, \hat{\mathbf{z}}_0^{(idx_{\text{last}})})$
    if $S < \tau$ **then**
      $\Delta L \leftarrow i - idx_{\text{last}}$ ▷ Calculate actual interval
      // Dynamic threshold adjustment
      if $\Delta L \geq L_{\text{avg}} + \delta_{up}$ **then**
        $\tau \leftarrow \tau - \Delta\tau$ ▷ Decrease threshold to relax selection
      else if $\Delta L \leq L_{\text{avg}} - \delta_{\text{down}}$ **then**
        $\tau \leftarrow \tau + \Delta\tau$ ▷ Increase threshold to tighten selection
      end if
      $\mathcal{K} \leftarrow \mathcal{K} \cup \{i\}$
      $idx_{\text{last}} \leftarrow i$
    end if
  end if
end for
$\mathcal{N} \leftarrow \{1, \ldots, N\} \setminus \mathcal{K}$
return $\mathcal{K}, \mathcal{N}$

D Discussions

D.1 Orthogonality to Intra-step Sparsity

A notable advantage of RhymeFlow is its orthogonality to existing intra-step sparse attention techniques. This orthogonality stems from operating on different dimensions of the computational workload. Methods like SVG [36] target **intra-step efficiency**, optimizing how tokens interact within a single attention computation, while our method addresses **inter-step efficiency**, determining which frames participate in the computation at each denoising step. This fundamental distinction allows the two approaches to be synergistically combined for enhanced acceleration performance.

To illustrate, consider an intra-step method like SAP that constructs a token-level sparse attention mask, $\mathbf{M}_{\text{SAP}}$. Concurrently, our framework generates a distinct frame-level mask, $\mathbf{M}_{\text{Rhyme}}$, which dictates the set of active and inactive frames for a given step. A hybrid, highly efficient attention mechanism can be formulated by computing the element-wise product of these two masks:

$$\mathbf{M}_{\text{combined}} = \mathbf{M}_{\text{Rhyme}} \odot \mathbf{M}_{\text{SAP}} \tag{44}$$

The resulting mask, $\mathbf{M}_{\text{combined}}$, imposes a hierarchical sparsity. It first prunes the extensive computations corresponding to skipped non-keyframes (as dictated by $\mathbf{M}_{\text{Rhyme}}$) and then further sparsifies the attention calculations among the remaining active frames based on the token-level patterns in $\mathbf{M}_{\text{SAP}}$. This fusion of inter-step and intra-step strategies unlocks compounded acceleration gains, paving the way for even more efficient video generation models.

Algorithm 2 RhymeFlow: Asynchronous Denoising Flow Scheduling

Require: Noisy latents $\{\mathbf{z}_T^{(i)}\}$, Timesteps T, Warm-up steps T_w, Mid-point T_{mid}, Progressive strides $\{n_{\text{small}}, n_{\text{large}}\}$.
Ensure: Fully denoised latents $\{\mathbf{z}_0^{(i)}\}$.

$\mathcal{K}, \mathcal{N} \leftarrow$ DynamicKeyframeSelection(...) ▷ Via Algorithm 1
for $t = T$ **down to** 1 **do**
  if $t > T - T_w$ **then** ▷ **Phase I: Synchronous Warm-up**
    $\{\mathbf{z}_{t-1}^{(i)}\}_{i=1}^{N_f} \leftarrow \text{FullDenoiseStep}(\{\mathbf{z}_t^{(i)}\}_{i=1}^{N_f})$ ▷ Full 3D Attention
  else ▷ **Phase II: Asynchronous Scheduling**
    // Progressive Stride Assignment (Eq. 2)
    $n_{\text{skip}} \leftarrow n_{\text{small}}$ if $t > T_{\text{mid}}$ else n_{large}
    // Identify active frames to be updated
    $\mathcal{I}_{\text{active}} \leftarrow \mathcal{K} \cup \{j \in \mathcal{N} \mid (T - T_w - t) \equiv 0 \pmod{n_{\text{skip}}}\}$
    for each frame $i \in \mathcal{I}_{\text{active}}$ **do**
      // Latent Trajectory Projection (Sec 3.2)
      $\mathcal{C} \leftarrow \{\mathbf{z}_t^{(k)}\}_{k \in \mathcal{K}} \cup \{\hat{\mathbf{z}}_t^{(j)}\}_{j \in \mathcal{N} \setminus \mathcal{I}_{\text{active}}}$ ▷ Anchor + Projected states
      $n_i \leftarrow 1$ if $i \in \mathcal{K}$ else n_{skip} ▷ Step size for key/non-keyframe
      $\mathbf{z}_{t-n_i}^{(i)} \leftarrow \text{Denoise}(\mathbf{z}_t^{(i)} \mid \mathcal{C})$ ▷ Heterogeneous update
    end for
    // Skipped non-keyframes $\{\mathbf{z}^{(j)}\}$ remain at state t until next rhythmic point.
  end if
end for
return $\{\mathbf{z}_0^{(i)}\}_{i=1}^{N_f}$

D.2 Analysis of Failure Cases

To explore the upper bound of our acceleration framework, we conducted stress tests with an ultra-aggressive skipping stride ($n_{\text{skip}} = 7$). As illustrated in Figure 6, such extreme skipping triggers noticeable temporal aliasing and "shimmering" effects, particularly in non-keyframe regions. This degradation occurs because the linear approximation provided by our Latent Trajectory Projection assumes a locally smooth ODE path; at very large strides, this assumption fails to capture high-frequency motion non-linearities, causing asynchronous frames to drift from the anchor keyframes. These failure cases validate that our default conservative scheduling ($n_{\text{small}} = 2, n_{\text{large}} = 3$) is essential for maintaining artifact-free temporal coherence.

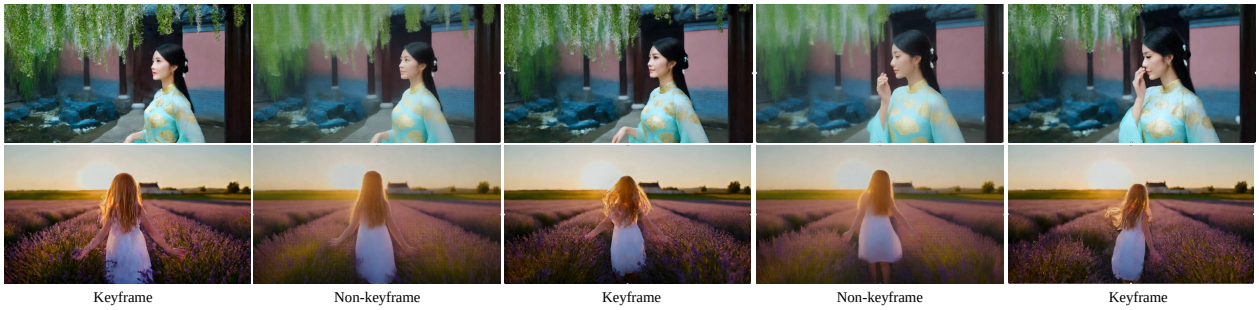


Figure 6 Visual failure cases under extreme acceleration ($n_{\text{skip}} = 7$). Aggressive skipping leads to shimmering artifacts and loss of high-frequency textures in non-keyframes, highlighting the necessity of our progressive scheduling balance.

D.3 Future Work

Several promising avenues exist for extending our work. A primary direction is the exploration of learned asynchronous scheduling. While our current progressive strategy is heuristic-based, future research could

employ a small policy network or uncertainty-based metric to dynamically determine which frames should be skipped and at what intervals, enabling a truly data-dependent acceleration.

Additionally, we envision the integration of RhymeFlow with post-training quantization (PTQ) and low-rank adaptations (LoRA). Since our method is strictly training-free and operates at the scheduling level, it remains orthogonal to model-level compression techniques. Combining these two paradigms could potentially unlock $3\times$ to $5\times$ speedups without compromising the generative capabilities of large-scale Video DiTs. Finally, investigating more sophisticated interpolation techniques beyond linear projection—such as utilizing optical flow priors—could further suppress artifacts at even larger skip strides.